

Bid Recommendation System

Codebase Architecture & Interconnectivity Map

Global Stat Solutions

April 2026 — Comprehensive Technical Reference

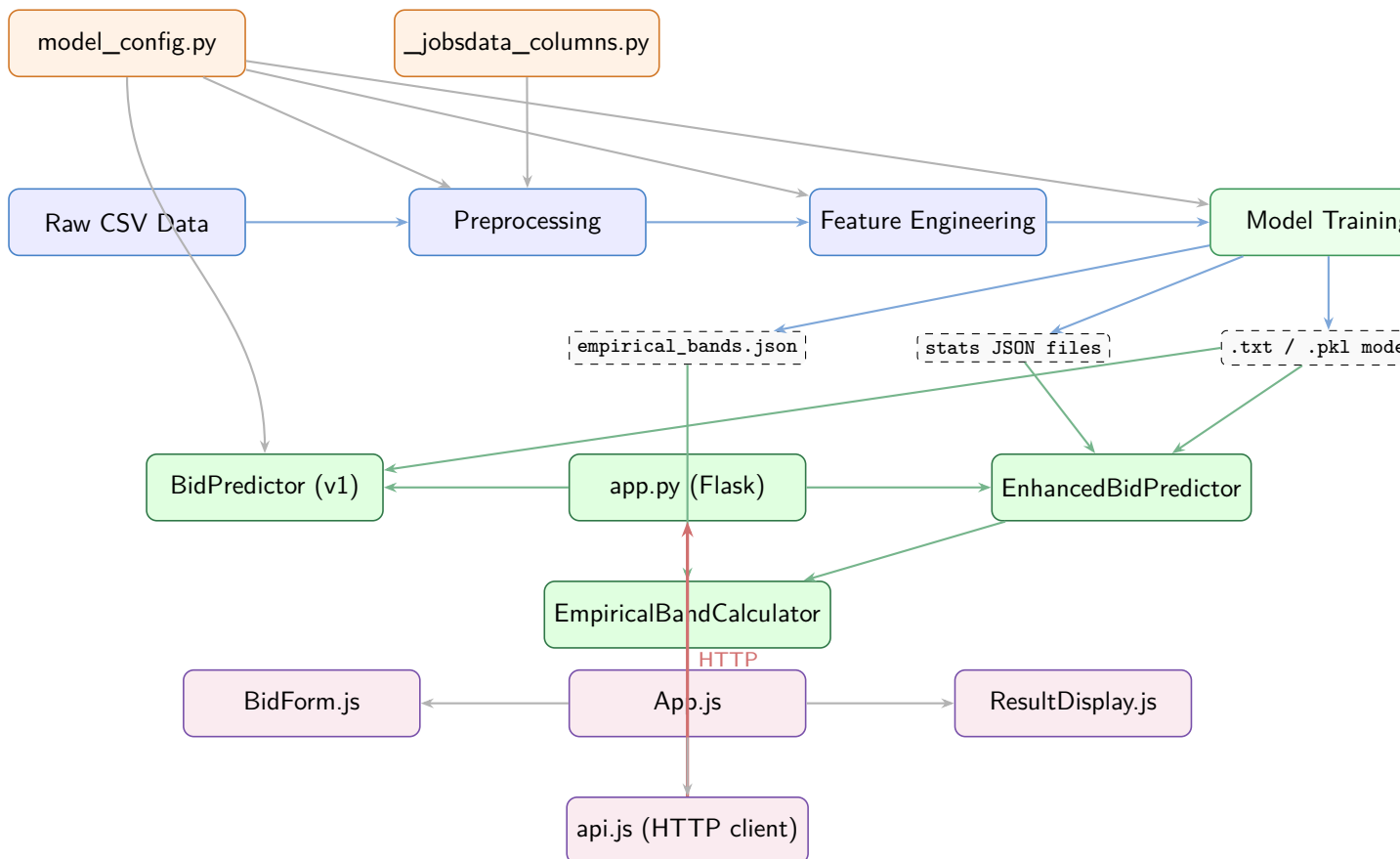
Contents

1	System-Level Network Graph	3
2	Codebase Inventory	3
3	Configuration Layer	4
3.1	config/model_config.py 269 lines	4
3.2	scripts/_jobsdata_columns.py 175 lines	5
4	Data Pipeline — Preprocessing	5
4.1	scripts/02_data_preprocessing.py 613 lines	5
4.2	scripts/21_jobsdata_preprocessing.py 445 lines	6
4.3	scripts/22_combined_data_preprocessing.py 359 lines	6
5	Data Pipeline — Feature Engineering	6
5.1	scripts/03_feature_engineering.py 591 lines	6
5.2	scripts/23_enhanced_feature_engineering.py 488 lines	7
6	Data Pipeline — Model Training	8
6.1	scripts/04_model_lightgbm.py 764 lines — Phase 1A v1	8
6.2	scripts/24_enhanced_bidfee_model.py 400 lines — Phase 1A v2	8
6.3	scripts/15_win_probability_baseline.py 525 lines — Phase 1B v1	8
6.4	scripts/25_enhanced_win_probability.py 473 lines — Phase 1B v2	9
7	Data Pipeline — Stats Generation	9
7.1	scripts/27_generate_api_stats_v2.py 202 lines	9
7.2	scripts/28_generate_zip_lookup.py 185 lines	9
8	API Layer — Flask Backend	9
8.1	api/app.py 486 lines — HTTP Router	9
8.2	api/prediction_service.py 900 lines — v1 Prediction Engine	10
8.3	api/enhanced_prediction_service.py 1076 lines — v2 Prediction Engine	10
8.3.1	Module-Level Constants & Functions	11
8.3.2	Class Methods	11
8.4	api/empirical_bands.py 312 lines — Confidence Intervals	12
9	v2 Prediction Request — Complete Data Flow	12
10	Frontend Layer — React	13
10.1	frontend/src/App.js 198 lines — State Container	13
10.2	frontend/src/services/api.js 170 lines — HTTP Client	14
10.3	frontend/src/components/BidForm.js 208 lines	14
10.4	frontend/src/components/ResultDisplay.js 543 lines	14
10.4.1	Helper Functions (lines 4–130)	14
10.4.2	FeeChart Component (lines 134–244)	15

10.4.3 Main ResultDisplay Component (lines 248–541)	15
10.5 Other Components	15
11 Cross-Module Import Map	16
11.1 Python Backend Imports	16
11.2 Frontend Component Tree	16
12 File Dependency Graph — What Reads What	16
13 Feature Engineering Detailed Breakdown	17
13.1 v1 Features (68 total) — 03_feature_engineering.py	17
13.2 v2 Features (44+ total) — 23_enhanced_feature_engineering.py	18
14 Tests	19
14.1 tests/test_validation.py (435 lines, 34 tests)	19
14.2 tests/test_models.py (292 lines)	19
15 Experiment Scripts	19
16 Runtime Artifact Inventory	19
17 Known Issues & Technical Debt	20

System-Level Network Graph

The entire system is a four-layer pipeline: **Configuration** feeds into an **offline ML training pipeline** (scripts/), which produces models and statistics consumed by a **real-time Flask API** (api/), which serves a **React frontend** (frontend/). Every arrow below represents an actual file read, import, or HTTP call.



Codebase Inventory

File	Lines	Layer	Classes / Functions
Configuration			
<code>config/model_config.py</code>	269	Config	0 classes, 0 functions
<code>scripts/_jobsdata_columns.py</code>	175	Config	0 classes, 1 list constant
ML Pipeline (scripts/)			
<code>01_eda.py</code>	886	EDA	0 classes, exploratory
<code>02_data_preprocessing.py</code>	613	Prep	1 class, 15 methods
<code>03_feature_engineering.py</code>	591	FE	1 class, 15 methods
<code>04_model_lightgbm.py</code>	764	Model	1 class, 12 methods
<code>15_win_probability_baseline.py</code>	525	Model	1 class, ~10 methods
<code>20_jobsdata_eda.py</code>	652	EDA	0 classes, exploratory
<code>21_jobsdata_preprocessing.py</code>	445	Prep	1 class, 7+ methods
<code>22_combined_data_preprocessing.py</code>	359	Prep	0 classes, 5 functions
<code>23_enhanced_feature_engineering.py</code>	488	FE	0 classes, 4 functions

File	Lines	Layer	Classes / Functions
24_enhanced_bidfee_model.py	400	Model	1 class, 6 methods
25_enhanced_win_probability.py	473	Model	pipeline script
26_enhanced_model_validation.py	222	Eval	comparison script
27_generate_api_stats_v2.py	202	Stats	0 classes, 2 functions
28_generate_zip_lookup.py	185	Stats	0 classes, 1 function
28_price_ratio_win_prob.py	501	Experiment	price_ratio model
29_fee_sensitive_win_prob.py	704	Experiment	LOO v3 model
30_cross_validation.py	247	Eval	0 classes, 2 functions
33_predict_demo.py	341	Demo	0 classes, 1 function
API Layer (api/)			
app.py	486	Router	0 classes, 12 route functions
prediction_service.py	900	Service	1 class, 12 methods
enhanced_prediction_service.py	1076	Service	1 class, 12 methods
empirical_bands.py	312	Utility	1 class, 6 methods
Frontend (frontend/src/)			
App.js	198	Container	1 component, 4 functions
services/api.js	170	HTTP	0 components, 8 functions
components/BidForm.js	208	Form	1 component
components/ResultDisplay.js	543	Display	2 components, 5 helpers
components/Header.js	20	UI	1 component
components/SegmentStats.js	88	UI	1 component
components/StateBenchmarks.js	112	UI	1 component
components/FeeSensitivityCharts.js	69	Chart	1 component
Tests			
tests/test_validation.py	435	Test	34 test methods
tests/test_models.py	292	Test	model-dependent tests
Total	~12,800		

Configuration Layer

config/model_config.py 269 lines

The single source of truth for all paths, hyperparameters, and feature exclusion lists. Every pipeline script and both API prediction services import from this file.

Constant	Line	Purpose
ROOT_DIR, DATA_DIR, ...	1–20	All filesystem paths
DATA_START_DATE		"2023-01-01" — v1 uses 2023+ data only
MIN_TRAINING_FEE		Minimum \$100 fee filter
EXCLUDE_COLUMNS		13 IDs/targets/dates excluded from features
JOBDATA_FEATURES_TO_EXCLUDE		26 JobData features that degrade performance
LEAKY_JOB_DERIVED_FEATURES		JobCount, IECount, LeaseCount, SaleCount
LIGHTGBM_CONFIG		Phase 1A hyperparameters: leaves=18, LR=0.02, depth=8
TRAIN_TEST_SPLIT		60/20/20 time-based ratios

Constant	Line	Purpose
PREDICTION_CONFIG		Runtime clamps: win_prob_min=0.05, win_prob_max=0.95
EVALUATION_METRICS		Regression + classification metric lists

Imported by: Every script (02–33), `prediction_service.py`, `enhanced_prediction_service.py`, `empirical_bands.py`.

`scripts/_jobsdata_columns.py` 175 lines

Maps 143 column indices to names for the headerless `JobsData.csv`. Contains `JOBSDATA_COLUMNS` list with columns grouped into: Core Job Info (0–7), Classification (8–14), Temporal (15–27), Geography (28–31), Property (32–52), Office (58–61), Client (62–66), and ZipCodeMaster Demographics (67–142).

Imported by: `21_jobsdata_preprocessing.py`, `22_combined_data_preprocessing.py`, `27_generate_api_s`, `28_generate_zip_lookup.py`.

Data Pipeline — Preprocessing

`scripts/02_data_preprocessing.py` 613 lines

`BidDataPreprocessor` L36 — Transforms raw `BidData` CSV into a clean dataset ready for feature engineering. Handles the entire v1 data cleaning pipeline.

Method	Line	What It Does
<code>__init__</code>	39	Sets paths, initializes <code>df_raw</code> , <code>df_clean</code>
<code>load_data</code>	61	Reads raw CSV, converts <code>BidDate</code> to datetime
<code>handle_duplicate_bidids</code>	84	Aggregates Master/SubJob rows: max BidFee, sum counts, first for categoricals
<code>filter_relevant_bids</code>	188	Keeps only Won/Lost/Placed; drops Active/Declined/Approved
<code>handle_missing_target_variable</code>	227	Removes rows with null BidFee
<code>handle_outliers_in_bidfee</code>	254	Caps BidFee at 99th percentile; saves original
<code>handle_missing_targettime</code>	293	Imputes TargetTime via group median (PropertyType × BidStatus)
<code>handle_missing_categorical_features</code>	346	Fills 15 categorical columns with “Unknown”
<code>handle_outliers_numerical_features</code>	388	Caps 6 numerical columns at 99th percentile
<code>create_temporal_features</code>	413	Extracts Year/Quarter/Month/DayOfWeek + cyclical sin/cos
<code>create_target_variable_for_classification</code>	439	Binary target: Won=1, else=0
<code>validate_data_quality</code>	459	Checks for nulls in critical columns
<code>save_processed_data</code>	496	Writes <code>BidData_processed.csv</code>
<code>generate_preprocessing_report</code>	513	Writes JSON report with row counts & quality stats
<code>run_full_preprocessing</code>	567	Orchestrates all steps in sequence

Reads: `data/processed/BidData_cleaned.csv`

Writes: `data/processed/BidData_processed.csv`, `outputs/reports/preprocessing_report.json`

scripts/21_jobsdata_preprocessing.py 445 lines

JobsDataPreprocessor L70 — Cleans the larger JobsData (215K rows, 2018+) for v2 training.

Method	Line	What It Does
<code>load_data</code>	77	Reads raw JobsData with JOBSDATA_COLUMNS headers
<code>filter_closed_recent</code>	94	Keeps Closed jobs from 2018 onward
<code>clean_netfee</code>	116	Removes null/zero fees; caps at 99th percentile
<code>handle_job_types</code>	149	Deduplicates by JobId, keeps max NetFee
<code>clean_categorical_features</code>	173	Collapses rare SubType/SpecificUse (<50 count) to “Other”; standardizes case
<code>clean_joblength</code>	230	Parses JobLength_Days, clamps to [1, 365], imputes by PropertyType median
<code>clean_numeric_features</code>	263	Converts “NULL” strings, caps at 99th percentile, fixes YearBuilt

Reads: `data/raw/JobsData.csv`

Writes: `data/processed/JobsData_processed.csv`

Imports: `_jobsdata_columns.py` for column names

scripts/22_combined_data_preprocessing.py 359 lines

Enriches BidData with JobsData features for v2 win probability training. Module-level functions (no class).

Function	Line	What It Does
<code>load_biddata</code>	70	Loads BidData_cleaned.csv, creates Won flag
<code>load_jobsdata</code>	89	Loads raw JobsData with column headers, cleans types
<code>enrich_won_bids</code>	126	LEFT JOIN on JobId: extracts SubType, GrossBuildingSF, OfficeRegion, demographics
<code>enrich_lost_bids</code>	184	Office/property-level medians: approximates JobsData values for bids that lost
<code>combine_and_finalize</code>	264	Concatenates Won + Lost, aligns columns, fills remaining NaN

Reads: `data/processed/BidData_cleaned.csv`, `data/raw/JobsData.csv`

Writes: `data/processed/BidData_enriched_v2.csv`

Important: Won bids get real JobsData values; lost bids get office/property medians. This *differential imputation* is a known leakage vector — the v2 win prob model excludes all 42 differentially-imputed features and their derivatives.

Data Pipeline — Feature Engineering**scripts/03_feature_engineering.py** 591 lines

BidFeatureEngineer L36 — Creates 68+ features from processed BidData for v1 models. All target-derived features use leave-one-out aggregation to prevent leakage.

Method	Line	Features	What It Creates
<code>create_rolling_features_by_group</code>	76	3	Rolling avg/std fee by segment & office (window=10)
<code>create_lag_features_by_client</code>	116	3	lag1/lag2 BidFee, lag1 Target-Time per client
<code>create_cumulative_client_features</code>	147	3	Expanding bids/wins/winrate per client
<code>create_aggregation_features</code>	179	15	LOO avg_fee, std_fee, win_rate for 5 groups
<code>create_competitiveness_features</code>	239	5	bid_vs_segment_ratio, fee_diff, fee_percentile
<code>create_temporal_behavioral_features</code>	285	9	days_since_last_bid, bid_count_30d, calendar features
<code>create_market_dynamics_features</code>	336	4	bid_density, combo_freq, specialization, workload
<code>create_risk_volatility_features</code>	378	4	segment CV, client consistency, targettime ratio
<code>create_interaction_features</code>	417	6	Cross-feature products (fee × time, etc.)
<code>create_categorical_encodings</code>	461	2	Frequency encoding for segment & property type

Total features created: 54 engineered + 14 temporal (from script 02) = 68

Reads: `data/processed/BidData_processed.csv`

Writes: `data/features/bid_features.csv`

`scripts/23_enhanced_feature_engineering.py` 488 lines

Creates v2 features for *both* Phase 1A (JobsData → NetFee) and Phase 1B (enriched BidData → Win). Uses shared `build_common_features` to ensure consistency.

Function	Line	What It Does
<code>leave_one_out_mean</code>	58	LOO aggregation: excludes current row from group mean
<code>frequency_encode</code>	65	Maps category → raw count (min_count=10)
<code>build_common_features</code>	71	Shared engine: LOO aggregates (segment, state, property type), rolling features, frequency encoding, JobLength features (log, bucket, × segment), building area (log, fee/sqft), interaction terms, temporal features, zip demographics
<code>engineer_phase1a_features</code>	265	Calls <code>build_common_features</code> on JobsData with NetFee target

Key v2 additions: SubType features (`subtype_avg_fee`, `subtype_frequency`), OfficeRegion features (`office_region_avg_fee`), CompanyLocation frequency, JobLength derivatives, building area features, zip demographics (15 columns), `income_x_segment_fee`, `pop_density_log`.

Reads: `data/processed/JobsData_processed.csv`, `data/processed/BidData_enriched_v2.csv`

Writes: `data/features/JobsData_features_v2.csv`, `data/features/BidData_features_v2.csv`

Data Pipeline — Model Training

scripts/04_model_lightgbm.py 764 lines — Phase 1A v1

`LightGBMBidFeePredictor` L57 — Trains the v1 bid fee regression model.

Method	Line	What It Does
<code>load_data</code>	96	Loads features CSV; filters to 2023+
<code>select_top_features</code>	178	Reduces to top 68 via <code>selected_features_top68.csv</code>
<code>prepare_features</code>	226	Excludes ID/target columns; selects numeric only
<code>time_based_split</code>	286	Chronological 60/20/20 split
<code>train_model</code>	350	Applies <code>log1p</code> transform; trains LightGBM with early stopping (1500 rounds)
<code>evaluate_model</code>	428	Inverts <code>expm1</code> ; computes RMSE (\$531), MAPE (15.7%), R^2 (0.94)
<code>feature_importance_analysis</code>	537	Top 30 features by gain; saves CSV + plot
<code>shap_analysis</code>	586	SHAP summary on test sample
<code>save_model</code>	631	Writes <code>.txt</code> model + metadata JSON + predictions CSV
<code>run_full_pipeline</code>	693	Orchestrates all steps

Reads: `data/features/bid_features.csv`, `outputs/reports/selected_features_top68.csv`

Writes: `outputs/models/lightgbm_bidfee_model.txt`, `outputs/models/lightgbm_metadata.json`

scripts/24_enhanced_bidfee_model.py 400 lines — Phase 1A v2

`EnhancedBidFeePredictor` L108 — Trains v2 bid fee model on JobsData (215K rows).

Method	Line	What It Does
<code>load_data</code>	117	Loads JobsData features; filters 2018+ and fee $\geq$ <code>MIN_TRAINING_FEE</code>
<code>prepare_features</code>	144	Excludes IDs, targets, categoricals, leaky features
<code>time_based_split</code>	162	Chronological 60/20/20
<code>train</code>	187	<code>log1p</code> transform; 3000 rounds, early stop=150
<code>evaluate</code>	218	MAPE 7.3%, R^2 0.924, overfit 2.09×
<code>feature_importance</code>	269	Saves importance CSV + plot

v2 hyperparameters: `leaves=18`, `LR=0.02`, `depth=6`, `reg_alpha=20.0`, `reg_lambda=20.0` (heavy regularization).

Writes: `outputs/models/lightgbm_bidfee_v2_model.txt`, `outputs/models/lightgbm_bidfee_v2_metadata.json`

scripts/15_win_probability_baseline.py 525 lines — Phase 1B v1

Trains v1 win probability classifier. Uses 72 features including BidFee as a direct input (the model learns fee-sensitivity from data). Excludes `LEAKY_JOB_DERIVED_FEATURES` and win-rate features.

Key metrics: AUC 0.870, accuracy 79.0%, overfit 1.09×

Writes: `outputs/models/lightgbm_win_probability.txt`, metadata JSON

scripts/25_enhanced_win_probability.py 473 lines — Phase 1B v2

Trains v2 win probability on enriched BidData (2018+). Excludes all 42 differentially-imputed Jobs_* features plus their derivatives (fee_per_sqft, building_sf_log, etc.) to prevent leakage.

Key metrics: AUC 0.948, accuracy 86.7%, overfit 1.04×, 44 features.

Known issue: Model is **saturated at inference** — raw output is 0.97–0.98 for all inputs. Bid-Fee is not in the top 10 features. `subtype_frequency` (31%) and `office_region_avg_fee` (25%) dominate. The API falls back to a heuristic sigmoid when this is detected.

Data Pipeline — Stats Generation

scripts/27_generate_api_stats_v2.py 202 lines

Pre-computes all runtime lookup tables consumed by the API at startup.

Function	Line	What It Produces
<code>generate_phase1a_stats</code>	30	Global stats, dropdown options, per-segment/state/property/subtype/office averages, rolling proxies, state coordinates, SubType-by-PropertyType cascading map
<code>generate_feature_defaults</code>	148	Per-feature global median/mean + segment-specific medians

Writes: `outputs/reports/api_precomputed_stats_v2.json`, `outputs/reports/feature_defaults_v2.json`

scripts/28_generate_zip_lookup.py 185 lines

`generate_zip_lookup` L58 — Builds a static dictionary mapping 20,040 zip codes to demographics (15 features), state, lat/lon, distance, and sample count. Only includes zips with ≥ 5 demographic values.

Writes: `outputs/reports/zip_demographics_lookup.json`

API Layer — Flask Backend

api/app.py 486 lines — HTTP Router

Flask application with CORS. Lazy-loads v1 and v2 predictors on first request.

Endpoint	Method	Line	Purpose
<code>/api/ping</code>	GET	38	Liveness check
<code>/api/debug</code>	GET	68	File status, init errors
<code>/api/health</code>	GET	139	Health check (Render uptime)
<code>/api/options</code>	GET	152	v1 dropdown data
<code>/api/predict</code>	POST	168	v1 prediction
<code>/api/segment/<name></code>	GET	239	v1 segment stats
<code>/api/batch-predict</code>	POST	256	v1 batch prediction
<code>/api/v2/options</code>	GET	314	v2 dropdown data (incl. subtypes, regions)

Endpoint	Method	Line	Purpose
/api/v2/predict	POST	329	v2 prediction (active)
/api/v2/zip/<code>	GET	380	Zip lookup + state resolution
/api/v2/segment/<name>	GET	414	v2 segment stats

Imports: `prediction_service.get_predictor`, `enhanced_prediction_service.EnhancedBidPredictor`

`api/prediction_service.py` 900 lines — v1 Prediction Engine

`BidPredictor` L49 — Full v1 inference pipeline.

Method	Line	What It Does
<code>__init__</code>	56	Loads v1 models, stats, defaults, bands, zip data
<code>_load_model</code>	76	Reads <code>lightgbm_bidfee_model.txt</code>
<code>_load_win_probability_model</code>	88	Reads <code>lightgbm_win_probability.txt</code> + metadata
<code>_compute_feature_statistics</code>	109	Loads <code>api_precomputed_stats.json</code> ; computes global/segment/state/property/office averages
<code>_load_empirical_bands</code>	223	Instantiates <code>EmpiricalBandCalculator</code> , loads <code>empirical_bands.json</code>
<code>_load_feature_defaults</code>	236	Loads <code>feature_defaults.json</code> , <code>state_coordinates.json</code> , <code>rolling_stats.json</code>
<code>_generate_features</code>	271	Maps 6 raw inputs → 68 features. 15 feature categories: raw inputs, segment/state/property/office aggregates, rolling, client history, competitiveness, temporal (sin/cos), market dynamics, risk, interactions, demographics
<code>predict</code>	506	Main entry. generate features → predict fee (<code>expm1</code>) → floor enforcement → confidence interval → inject fee as BidFee → predict win prob → calculate EV → build recommendation
<code>predict_win_probability</code>	729	Builds win prob feature vector; predicts with v1 classifier; clamps [0.05, 0.95]
<code>_fallback_win_probability</code>	798	Sigmoid heuristic: $P = 1/(1 + e^{5(\text{ratio}-1)})$ scaled to [0.20, 0.75]
<code>get_dropdown_options</code>	711	Returns segments, property types, states, offices
<code>get_segment_stats</code>	720	Returns per-segment avg/std/winrate/count

Runtime files loaded: `lightgbm_bidfee_model.txt`, `lightgbm_win_probability.txt`, `api_precomputed_stats.json`, `feature_defaults.json`, `state_coordinates.json`, `rolling_stats.json`, `empirical_bands.json`

`api/enhanced_prediction_service.py` 1076 lines — v2 Prediction Engine

`EnhancedBidPredictor` L158 — The production-active v2 engine with fee sensitivity curves, EV optimization, saturation detection, and heuristic fallback.

Module-Level Constants & Functions

Name	Line	Description
LOCATION_TO_REGION	47	Dict mapping 30+ office location names → 6 regions
SATURATION_THRESHOLD	134	0.93 — raw model output above this triggers heuristic
CURVE_FLAT_THRESHOLD_PP	135	5.0 — curve spread below this (pp) is replaced with heuristic
<code>_heuristic_win_prob(fee, blended)</code>	138	Logistic fallback: $P = 0.22 + \frac{0.58}{1+e^{3.5(r-1.0)}}$ where $r = \text{fee}/\text{blended}$

Class Methods

Method	Line	What It Does
<code>__init__</code>	161	Loads v2 models, stats, defaults, zip lookup, bands; runs startup canary
<code>_load_models</code>	179	Reads v2 .txt models + isotonic calibrator pickle
<code>_load_stats</code>	212	Reads <code>api_precomputed_stats_v2.json</code>
<code>_load_zip_lookup</code>	236	Reads <code>zip_demographics_lookup.json</code> (20K zips)
<code>_run_startup_canary</code>	246	Tests v2 win prob at 3 fees (\$500/\$3K/\$10K); logs WARNING if saturated
<code>_generate_features</code>	296	Maps raw inputs + zip → 44+ features. Segment, state, property, subtype, office region, company location, rolling, interactions, temporal (sin/cos), geographic (lat/lon from zip), JobLength (log, bucket), building area, 15 zip demographics, distance
<code>predict</code>	476	Main entry. Derives office_region → parse date → validate zip → generate features → predict fee (<code>expm1</code>) → benchmark floor → rare-combo blending → confidence interval → blended benchmark → inject fee → predict win prob (with saturation detection) → EV → fee sensitivity curve (20 points, 0.4× to 2.0×) → EV optimization → solid win ceiling (30% threshold) → build recommendation → warnings
<code>_predict_win_probability</code>	730	Builds vector; predicts raw; if raw ≥ 0.93: returns heuristic ; else applies isotonic calibration; clamps [0.05, 0.95]
<code>get_fee_sensitivity_curve</code>	835	Sweeps 20 fee points; calls <code>_predict_win_probability</code> for each; if flat (<5pp spread): replaces entire curve with heuristic
<code>_build_recommendation</code>	898	5 fee tiers × 3 win tiers = 15 headline combinations; adds market context, strategy tips, low-confidence notes, frontend signal

Method	Line	What It Does
<code>get_dropdown_options</code>	1058	v2 options including sub-types_by_property_type cascading map

Runtime files loaded: `lightgbm_bidfee_v2_model.txt`, `lightgbm_win_probability_v2.txt`, `win_probability_v2_calibrator.pkl`, `api_precomputed_stats_v2.json`, `feature_defaults_v2.json`, `zip_demographics_lookup.json`, `empirical_bands.json`

`api/empirical_bands.py` 312 lines — Confidence Intervals

`EmpiricalBandCalculator` L27 — Computes quantile-based confidence intervals from historical prediction residuals, stratified by fee bucket, segment, and state.

Method	Line	What It Does
<code>compute_bands_from_predictions</code>	45	Computes residual quantiles: global, by fee bucket (4 buckets), by segment (≥ 30 samples), by state (≥ 50 samples)
<code>_compute_quantiles</code>	95	Returns p5, p10, p25, p50, p75, p90, p95, std, n for a residual array
<code>get_confidence_interval</code>	109	Fallback hierarchy: segment $\rightarrow$ state $\rightarrow$ fee bucket $\rightarrow$ global. Selects quantile pair by confidence level (80% $\rightarrow$ p10/p90). Enforces $\geq \$500$ floor and $\geq \$100$ width
<code>save_bands</code> / <code>load_bands</code>	201/219	JSON serialization to/from <code>empirical_bands.json</code>

Fee buckets: low (0–2K), medium (2K–4K), high (4K–6K), very_high (6K+).

Imported by: `prediction_service.py`, `enhanced_prediction_service.py`

v2 Prediction Request — Complete Data Flow

This traces a single POST `/api/v2/predict` request from HTTP arrival to JSON response.

1. `app.py:329` — Flask receives POST. Extracts 7 fields from JSON body.
2. `app.py:364` — Calls `enhanced_predictor.predict(...)`
3. `enhanced_prediction_service.py:493` — Derives `office_region` from `LOCATION_TO_REGION` dict.
4. `:496` — Parses `open_date` string to datetime for temporal features.
5. `:508` — Validates `zip_code` against `self.zip_lookup`; detects state mismatch.
6. `:518` — Calls `_generate_features()` $\rightarrow$ builds 44+ feature dict from raw inputs + precomputed stats + zip demographics.
7. `:534` — Orders features to match `self.model_features` (training metadata order). Missing features filled from `self.feature_defaults`.
8. `:546` — Phase 1A prediction: `model.predict(X)[0]` $\rightarrow$ `expm1()` $\rightarrow$ dollar amount.
9. `:551` — Benchmark-aware floor: `max($500, 30%  $\times$  segment_avg)`.

10. **:558** — Rare-combo blending: if sample count < 100, blends prediction toward segment average.
11. **:567** — `EmpiricalBandCalculator.get_confidence_interval()` → 80% CI with segment → state → bucket → global fallback.
12. **:597** — Computes blended benchmark: $0.4 \times \text{seg} + 0.3 \times \text{state} + 0.3 \times \text{prop}$.
13. **:604** — Injects predicted fee as `BidFee` into features dict. Calls `_predict_win_probability()`.
14. **:771** — Inside `_predict_win_probability`: raw model output. If ≥ 0.93 (saturated), returns `_heuristic_win_prob()` instead.
15. **:610** — $\text{EV} = P(\text{Win}) \times \text{predicted_fee}$.
16. **:613** — `get_fee_sensitivity_curve()` sweeps 20 points from $0.4\times$ to $2.0\times$ predicted fee. Each point calls `_predict_win_probability`. If curve is flat (<5pp), replaces with heuristic.
17. **:624** — EV optimization: finds fee point on curve that maximizes $P(\text{Win}) \times \text{fee}$.
18. **:630** — Solid win ceiling: highest fee with $\geq 30\%$ win probability.
19. **:634** — If EV-optimal fee > ceiling, caps at ceiling; sets `ev_capped_at_ceiling = true`.
20. **:658** — `_build_recommendation()` generates plain-language headline, detail, strategy tip, and signal.
21. **:690** — Returns complete JSON response with 15+ fields.

Frontend Layer — React

`frontend/src/App.js` 198 lines — State Container

Root component. All state lifted here; children are presentational.

State Variable	Line	Purpose
<code>options</code>	9	Dropdown data from GET <code>/v2/options</code>
<code>loading</code>	18	Initial options fetch in flight
<code>predicting</code>	19	Prediction API call in flight
<code>prediction</code>	20	Full API response object
<code>error</code>	21	Error message string
<code>formData</code>	24	User input: <code>segment</code> , <code>property_type</code> , <code>state</code> , <code>turnaround_days</code> , <code>zip_code</code> , <code>sub_property_type</code> , <code>office_location</code> , <code>open_date</code>
<code>zipStatus</code>	34	Zip lookup result: <code>{found, state, loading}</code>

Function	Line	What It Does
<code>loadOptions</code>	38	Calls <code>fetchV2Options()</code> , sets dropdown defaults
<code>handleInputChange</code>	65	Updates <code>formData</code> ; resets subtype on <code>property_type</code> change; triggers zip lookup at 5 digits
<code>handleSubmit</code>	96	Builds payload (splits <code>turnaround_days</code> into <code>target_time</code> + <code>delivery_days</code>); calls <code>predictV2BidFee()</code>

Function	Line	What It Does
<code>handleReset</code>	127	Clears prediction and error

API calls: `fetchV2Options` (mount), `lookupZip` (on zip input), `predictV2BidFee` (on submit).

`frontend/src/services/api.js` 170 lines — HTTP Client

All backend communication via native Fetch API.

Function	Line	Method	Endpoint
<code>fetchOptions</code>	11	GET	<code>/api/options</code>
<code>predictBidFee</code>	30	POST	<code>/api/predict</code>
<code>fetchSegmentStats</code>	55	GET	<code>/api/segment/{name}</code>
<code>batchPredict</code>	74	POST	<code>/api/batch-predict</code>
<code>fetchV2Options</code>	99	GET	<code>/api/v2/options</code>
<code>predictV2BidFee</code>	118	POST	<code>/api/v2/predict</code>
<code>lookupZip</code>	143	GET	<code>/api/v2/zip/{code}</code>
<code>checkHealth</code>	162	GET	<code>/api/health</code>

`API_BASE_URL` L6: `process.env.REACT_APP_API_URL || '/api'`. Production uses Vercel env var pointing to Render backend.

`frontend/src/components/BidForm.js` 208 lines

Pure presentational form. Receives `formData`, `options`, `onChange`, `onSubmit`, `loading`, `zipStatus` as props. Contains 8 input fields (`segment`, `property_type`, `sub_property_type`, `zip_code`, `state`, `turnaround_days`, `open_date`, `office_location`) plus turnaround preset buttons (14/21/30/45/60 days).

No state, no API calls. All logic delegated to parent `App.js`.

`frontend/src/components/ResultDisplay.js` 543 lines

The most complex frontend component. Contains helper functions, an inner `FeeChart` component, and the main `ResultDisplay` component.

Helper Functions (lines 4–130)

Function	Line	What It Does
<code>fmt(n)</code>	6	Formats number with locale separators (3245 → “3,245”)
<code>interpolateWinProb(fee, curve)</code>	10	Linear interpolation on fee curve to find win prob at any fee
<code>winLabel(pct)</code>	24	Maps % to label: “Strong” / “Moderate” / “Low”
<code>winColor(pct)</code>	31	Maps % to hex: green ($\geq 55\%$) / amber ($\geq 35\%$) / red ($< 35\%$)

Function	Line	What It Does
<code>buildContextMessage(...)</code>	39	Client-side contextual messaging engine. 6 cases: EV-capped, flat curve, low win prob, low confidence, strong odds, moderate default. Returns {headline, body, tip, signal}

FeeChart Component (lines 134–244)

SVG-based interactive fee vs. win probability chart.

State: `chartFee` (slider position). **Props:** `curvePoints`, `recFee`, `maxFee`, `floorFee`, `evCapped`.

Renderers: Y-axis gridlines, fee curve polyline, 30% viability threshold (red dashed), floor/ceiling vertical markers, draggable slider, live stats (fee, win%, EV).

Main ResultDisplay Component (lines 248–541)

Key computed values (lines 248–311):

```

floorFee = confidence_interval.low
recFee = ev_optimal_fee || predicted_fee
maxFee = solid_win_ceiling || confidence_interval.high
evCapped = ev_capped_at_ceiling || false
winProbPct = interpolateWinProb(recFee, curvePoints)
evAtRec = (winProbPct / 100) * recFee
vsMarket = ((recFee - segment_benchmark) / segment_benchmark * 100)
isFlatCurve = (max_prob - min_prob) < 8

```

Render sections:

1. Warnings (317–322): first entry from `prediction.warnings[]`
2. Bid Range panel (325–404): floor / optimal / ceiling with scale bar
3. Win probability bar (407–420): colored bar + EV display
4. Flat curve note (425–434): shown if price has little effect
5. Contextual insight card (437–445): from `buildContextMessage`
6. Fee chart (448–465): expandable `FeeChart`
7. Market context strip (468–485): segment, state, zip, vs. segment %
8. Rush callout (488–498): if turnaround ≤ 21 days
9. Details panel (501–537): segment/state benchmarks, subtype/region effects

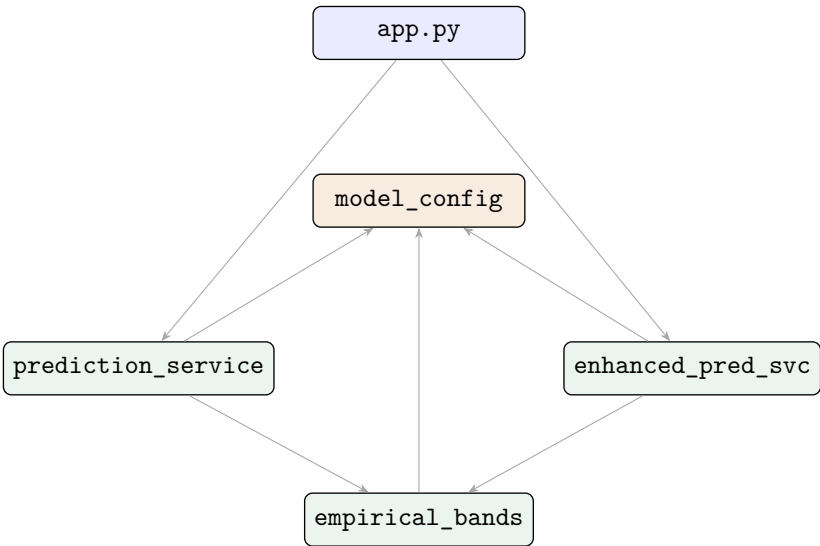
Other Components

File	Lines	Component	Purpose
<code>Header.js</code>	20	<code>Header</code>	Static brand header
<code>SegmentStats.js</code>	88	<code>SegmentStats</code>	Dropdown + stats grid; calls <code>fetchSegmentStats</code> API
<code>StateBenchmarks.js</code>	112	<code>StateBenchmarks</code>	State comparison bars (hard-coded <code>STATE_DATA</code>)

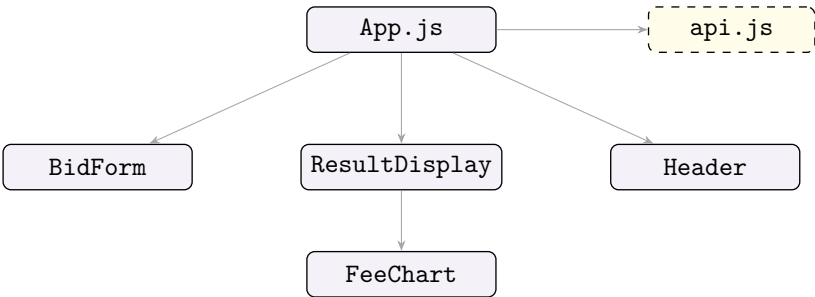
File	Lines	Component	Purpose
FeeSensitivityCharts.js	69	FeeSensitivityCharts	Recharts LineChart for fee curve (alternative to SVG FeeChart)

Cross-Module Import Map

Python Backend Imports



Frontend Component Tree



File Dependency Graph — What Reads What

Every runtime and training-time file dependency in the system:

Consumer	Direction	Producer
Training Pipeline		
02_data_preprocessing	reads	data/processed/BidData_cleaned.csv
02_data_preprocessing	writes	data/processed/BidData_processed.csv
03_feature_engineering	reads	BidData_processed.csv
03_feature_engineering	writes	data/features/bid_features.csv
04_model_lightgbm	reads	bid_features.csv
04_model_lightgbm	writes	lightgbm_bidfee_model.txt
15_win_prob_baseline	reads	bid_features.csv

Consumer	Direction	Producer
15_win_prob_baseline	writes	lightgbm_win_probability.txt
21_jobsdata_preproc	reads	data/raw/JobsData.csv
21_jobsdata_preproc	writes	data/processed/JobsData_processed.csv
22_combined_preproc	reads	BidData_cleaned.csv + JobsData.csv
22_combined_preproc	writes	BidData_enriched_v2.csv
23_enhanced_fe	reads	JobsData_processed.csv + BidData_enriched_v2.csv
23_enhanced_fe	writes	JobsData_features_v2.csv + BidData_features_v2.csv
24_enhanced_bidfee	reads	JobsData_features_v2.csv
24_enhanced_bidfee	writes	lightgbm_bidfee_v2_model.txt
25_enhanced_winprob	reads	BidData_features_v2.csv
25_enhanced_winprob	writes	lightgbm_win_probability_v2.txt + calibrator.pkl
27_api_stats_v2	reads	JobsData_features_v2.csv + model metadata
27_api_stats_v2	writes	api_precomputed_stats_v2.json + feature_defaults_v2.json
28_zip_lookup	reads	data/raw/JobsData.csv
28_zip_lookup	writes	zip_demographics_lookup.json
API Runtime		
BidPredictor.__init__	loads	lightgbm_bidfee_model.txt
BidPredictor.__init__	loads	lightgbm_win_probability.txt + metadata
BidPredictor.__init__	loads	api_precomputed_stats.json
BidPredictor.__init__	loads	feature_defaults.json + state_coordinates.json + rolling_stats.json
EnhancedBidPredictor.__init__	loads	lightgbm_bidfee_v2_model.txt
EnhancedBidPredictor.__init__	loads	lightgbm_win_probability_v2.txt + calibrator
EnhancedBidPredictor.__init__	loads	api_precomputed_stats_v2.json
EnhancedBidPredictor.__init__	loads	feature_defaults_v2.json
EnhancedBidPredictor.__init__	loads	zip_demographics_lookup.json
EmpiricalBandCalculator	loads	empirical_bands.json
Frontend → API (HTTP)		
App.js	GET	/api/v2/options
App.js	GET	/api/v2/zip/{code}
App.js	POST	/api/v2/predict
SegmentStats.js	GET	/api/segment/{name}

Feature Engineering Detailed Breakdown

v1 Features (68 total) — 03_feature_engineering.py

Category	Count	Method (Line)	Features Created
Rolling	3	<code>create_rolling</code> (76)	rolling_avg_fee_segment, rolling_avg_fee_office, rolling_std_fee_segment
Client Lag	3	<code>create_lag</code> (116)	lag1/lag2_bidfee_client, lag1_targettime_client
Cumulative	3	<code>create_cumulative</code> (147)	cumulative_bids/wins/winrate_client
LOO Aggregation	15	<code>create_aggregation</code> (179)	{segment, client, office, propertytype, state} × {avg_fee, std_fee, win_rate}
Competitiveness	5	<code>create_competitiveness</code> (239)	bid_vs_segment_ratio, bid_vs_client/state_ratio, fee_diff, fee_percentile
Temporal	9	<code>create_temporal</code> (285)	days_since_last_bid, bid_count_30d, calendar flags
Market Dynamics	4	<code>create_market</code> (336)	bid_density, combo_freq, specialization, workload
Risk / Volatility	4	<code>create_risk</code> (378)	segment_cv, client_consistency, targettime_ratio, fee_range
Interactions	6	<code>create_interaction</code> (417)	fee×time, client×std, office×state, etc.
Categorical	2	<code>create_categorical</code> (461)	segment_frequency, Property- Type_frequency
Temporal (script 02)	14	<code>create_temporal</code> (413)	Year, Quarter, Month, DayOfWeek + sin/cos encodings

v2 Features (44+ total) — 23_enhanced_feature_engineering.py

The v2 pipeline adds these feature categories on top of the v1 base:

New Category	Count	Features
SubType	2	subtype_avg_fee, subtype_frequency
Office Region	2	office_region_avg_fee, office_region_frequency
Company Location	1	company_location_frequency
JobLength	4	JobLength_Days, joblength_log, joblength_bucket, joblength_x_segment_fee
Building Area	4	GrossBuildingSF, building_sf_log, land_acres_log, fee_per_sqft
Zip Demographics	17	15 raw Zip_* features + income_x_segment_fee + pop_density_log

Tests

`tests/test_validation.py` (435 lines, 34 tests)

Runs in CI without model files. Tests config integrity, feature lists, data conventions, leakage guards, and prediction service logic.

`tests/test_models.py` (292 lines)

Requires trained model files on disk. Tests model loading, prediction shape, feature order matching, and metric thresholds.

Experiment Scripts

Scripts numbered 28+ (`price_ratio`), 29 (fee-sensitive v3), 30 (cross-validation), 32 (overfitting experiments), 40–41 (XGBoost alternatives), and 50 (BidFee model experiments) are experiment/research scripts. They produce analysis artifacts but do **not** feed into the production pipeline.

Script	Lines	Experiment
<code>28_price_ratio_win_prob.py</code>	501	Adds <code>price_ratio</code> feature to win prob model. Result: contributes only 0.96% importance. Not deployed.
<code>29_fee_sensitive_win_prob.py</code>	704	LOO expanding-window win rates replace frequency counts. Result: AUC drops to 0.871, fee features only 4.6%. Non-monotonic curve. Not deployed.
<code>30_cross_validation.py</code>	247	Time-series expanding-window CV for both models
<code>32_overfitting_experiments.py</code>	376	Regularization sweeps to reduce overfitting ratio
<code>40_xgboost_bidfee_model.py</code>	391	XGBoost alternative for bid fee regression
<code>41_xgboost_win_probability.py</code>	434	XGBoost alternative for win probability
<code>50_bidfee_model_experiments.py</code>	367	Log-transform experiments that led to v1 improvement

Runtime Artifact Inventory

All files in `outputs/` required at runtime by the API:

File	Format	Loaded By
<code>models/lightgbm_bidfee_model.txt</code>	LightGBM text	BidPredictor
<code>models/lightgbm_metadata.json</code>	JSON	BidPredictor
<code>models/lightgbm_win_probability.txt</code>	LightGBM text	BidPredictor
<code>models/lightgbm_win_probability_metadata.json</code>	JSON	BidPredictor
<code>models/lightgbm_bidfee_v2_model.txt</code>	LightGBM text	EnhancedBidPredictor
<code>models/lightgbm_bidfee_v2_metadata.json</code>	JSON	EnhancedBidPredictor
<code>models/lightgbm_win_probability_v2.txt</code>	LightGBM text	EnhancedBidPredictor
<code>models/lightgbm_win_probability_v2_metadata.json</code>	JSON	EnhancedBidPredictor

File	Format	Loaded By
<code>models/win_probability_v2_calibration.pickle</code>	Pickle	EnhancedBidPredictor
<code>reports/api_precomputed_stats.json</code>	JSON	BidPredictor
<code>reports/api_precomputed_stats_v2.json</code>	JSON	EnhancedBidPredictor
<code>reports/empirical_bands.json</code>	JSON	EmpiricalBandCalculator
<code>reports/feature_defaults.json</code>	JSON	BidPredictor
<code>reports/feature_defaults_v2.json</code>	JSON	EnhancedBidPredictor
<code>reports/state_coordinates.json</code>	JSON	BidPredictor
<code>reports/rolling_stats.json</code>	JSON	BidPredictor
<code>reports/zip_demographics_lookup.json</code>	JSON	EnhancedBidPredictor

Known Issues & Technical Debt

1. **Win prob v2 model saturation.** Raw output is 0.97–0.98 for all inputs. BidFee is not in the top 10 features by gain. The heuristic sigmoid fallback (`enhanced_prediction_service.py:138`) is active in production. The proper fix (v3 retraining with verified fee sensitivity) has been attempted (scripts 28, 29) but neither produced a viable model — the training data has fundamentally weak fee-to-outcome signal.
2. **StateBenchmarks hardcoded.** `StateBenchmarks.js` uses a `STATE_DATA` constant instead of API data. Should be replaced with a `/api/v2/state/{name}` endpoint.
3. **v2 bid fee overfit ratio 2.09×**. Slightly above the 2.0× target. Heavy regularization (`alpha=20`, `lambda=20`) keeps it near threshold.
4. **Frontend build incompatibility.** Node.js v25.4.0 is incompatible with Create React App (`shebang-regex` module). Builds fine on Vercel with a compatible Node version.
5. **Differential imputation leakage risk.** Won bids get real JobsData values, lost bids get medians. All 42 such features are excluded from win prob v2, but new features derived from these must also be excluded.